

LOG AI -- Security Log Processing & Threat Detection Platform

Comprehensive Project Implementation & Presentation Report

> Document Purpose: Technical architecture walkthrough, active implementation record, landing page interaction specification, and hackathon presentation (PPT) preparation guide based on the current codebase of LOG AI.

1. Project Overview (Executive Summary)

What Does LOG AI Do?

LOG AI is a high-performance security log ingestion, normalization, and explainable threat detection platform. It collects raw system logs from heterogeneous network perimeter appliances--such as Cisco ASA firewalls, Fortinet FortiGate gateways, Suricata IDS/IPS sensors, and pfSense appliances--converts them into a standardized common schema (OCSF 1.1), detects suspicious activity using an in-memory Isolation Forest ML model, correlates related events into incident clusters, and presents plain-English explanations and step-by-step mitigation commands to security operators.

Core Problem Solved

1. **Log Format Babel**: Security teams deal with dozens of incompatible log formats (CEF, Syslog, CSV, KV-pairs, JSON), making automated threat correlation across vendor siloes nearly impossible.

2. **Machine**

Primary Workflow

[Edge Log Payload]

2. Problem Statement

Enterprise Security Challenges Addressed

* **Heterogeneous Perimeter Appliance Ingestion**: Modern enterprise SOCs operate multi-vendor environments. Parsing firewall logs, IDS alerts, and network telemetry usually requires writing brittle, custom regex rules per device type.

* **
guaran

3. Solution Overview & Data Flow

LOG AI implements an end-to-end 6-stage telemetry processing architecture:

+-----+

| 1. P

4. How the System Works (End-to-End Walkthrough)

- 1. **Ingestion Request** (`app/routers/ingest.py`):

An H

5. Technical Architecture & Technology Stack

| Component | Technology | Purpose & Justification |

| :--- |

6. Landing Page Interaction Architecture (`LandingPage.jsx`)

The landing page features a section-level scroll-driven interaction model built on strict architectural hierarchy:

MAJOR SECTION

1. Viewport-Aware Section Activation (`IntersectionObserver`)

* Each major storytelling section (`6-Stage Log Processing Pipeline`, `Log Format Capabilities`, `End-to-End System Topology`, `SOC Efficiency Estimator`) owns an independent `IntersectionObserver`.

* A s

2. Section-Scoped Scroll Progress Calculation

* Scroll progress is computed relative strictly to the active section's bounding rectangle (`getBoundingClientRect()`).

* As
thresh

3. Multi-Input Precedence Model (Hover + Click + Touch + Keyboard)

Subtopic selection follows a priority precedence chain:

* **Hover** (`onMouseEnter/onMouseLeave`): Temporarily previews the hovered subtopic and updates the main visualization and technical explanation panel. Moving the pointer away returns to the selected or scroll-active state.

4. Central Morphing Visualizations Across Major Sections

* **6-Stage Log Processing Pipeline** (`#pipeline`): Features a horizontal 6-stage node flow bar, active stage progress meter, raw syslog stream, regex format identification, extracted key-value fields, OCSF 1.1 JSON schemas, Isolation Forest threat score gauge, and 3-step firewall mitigation playbooks (`iptables -A INPUT...`).

5. Dual Themes & Reduced Motion Support

* Both **Sage Green** (off-white editorial light theme) and **Cyber Void** (dark slate theme) share 100% identical interaction behavior.

7. Unified SOC Dashboard & Core Pages

1. Unified SOC Dashboard (`DashboardPage.jsx`)

* Real-time visibility into overall threat posture, active incident count, pipeline latency, and live telemetry feeds.

2. Log Explorer (`LogExplorerPage.jsx`)

* Multi-parameter log search and filtering interface across vendor parsers with severity badges, IP filtering, and JSON syntax rendering.

3. Forensic Investigation & Merkle Audit (`ForensicsPage.jsx`)

* Re-calculates SHA-256 cryptographic hashes of raw log payloads against the Merkle tree root hash to confirm zero data tampering. Reads compressed Gzip records from `data/raw/`.

4. Rule Studio (`RuleStudioPage.jsx`)

* Rule configuration page for custom threat thresholds, pattern triggers, and score multipliers evaluated by `rules_engine.py`.

5. AI Copilot (`CopilotWidget.jsx` & `explainer.py`)

* Provides plain-English explanations of flagged anomalies and step-by-step mitigation commands for active security

incidents.

6. Authentication Pages (`LoginPage.jsx` & `RegisterPage.jsx`)

* Operator sign-in and account creation forms featuring brand logo integration (`LOG // AI`) and an interactive **view password visibility toggle** (eye button).

8. Supported Log Formats

LOG AI includes native parser support for top enterprise security perimeter appliances:

Appliance / Format	Match Pattern / Signature	Extracted Fields	Standardized event_type	
				:---

9. OCSF 1.1 Normalization Schema

All parsed logs are mapped to the standard UnifiedEvent Pydantic model (app/normalization/schema.py):

```
class UnifiedEvent(BaseModel):
```

```
t:
```

10. Detection & Anomaly Analysis (Isolation Forest ML)

Pure NumPy Isolation Forest Engine (`app/detection/engine.py`)

LOG AI features a custom, pure Python/NumPy implementation of the Isolation Forest anomaly detection algorithm.

1. **Feature Vector Extraction**:

2. **Isolation Tree Ensemble**:

3. **Feature Attribution Z-Scores**:

11. Severity & Incident Correlation

Severity Threshold Mapping

* **CRITICAL** (\$\ge 90.0\$ Threat Score): Immediate threat requiring active mitigation.

Incident Clustering Heuristic (`app/detection/correlation.py`)

* **Grouping Key**: Scoped by `source\_ip`.

12. Offline GeoIP Resolution (`app/services/geoip.py`)

LOG AI provides zero-network-latency offline IP geolocation:

13. Security, Authentication & Multi-Tenancy

* **Authentication Endpoint (`app/routers/auth.py`)**: OAuth2 password bearer tokens using JWT (JSON Web Tokens) with 24-hour expiration.

14. Traditional SIEM vs. LOG AI Comparison

| Feature / Capability | Traditional Legacy SIEM | LOG AI Platform |

15. Presentation Slide Material (PPT Content)

This section provides a 12-slide presentation structure for hackathon demos:

Slide 1 -- Title Slide

* **Title**: LOG AI -- Security Log Processing & Threat Detection Platform

Slide 2 -- The Problem: Enterprise Log Chaos

* **Key Points**:

Slide 3 -- The Solution: LOG AI Architecture

* **Key Points**:

Slide 4 -- Zero-Loss Ingestion & Forensic Integrity

* **Key Points**:

Slide 5 -- Multi-Vendor Parsing & OCSF 1.1 Schema

* **Key Points**:

* Nat

Slide 6 -- Pure NumPy Isolation Forest Anomaly Detection

* **Key Points**:

* 5-fe

Slide 7 -- 15-Minute Sliding Window Correlation

* **Key Points**:

* Gro

Slide 8 -- Explainable AI (XAI) & Actionable Playbooks

* **Key Points**:

* Z-s

Slide 9 -- Offline GeoIP & Threat Map

* **Key Points**:

* 100

Slide 10 -- Financial Impact & SOC ROI Estimator

* **Key Points**:

* Dua

Slide 11 -- Live System Demo & Technology Stack

* **Key Points**:

* Tec

Slide 12 -- Summary & Future Roadmap

* **Key Points**:

* Cor

16. Live Demo Script (3-Minute Sequence)

- 1. 0:00 - 0:45 | Landing Page & Interactive Vector Radar:

* *Ac

17. Likely Judge Technical Questions & Answers

- 1. Q: Why implement a custom Isolation Forest instead of using Scikit-Learn?

* *A

enviro

18. Current Technical Limitations & Future Scope

Current Limitations

* Correlation Scope: Incident correlation is currently scoped per `source\_ip`. It does not yet group attacks across distributed botnet subnets sharing different source IPs.

* *S

cluster

Future Scope

Automated Active Defense: Adding direct API connectors to push firewall block rules directly to Cisco ASA and Fortinet routers upon incident creation.

* **
 Redis

19. Implementation File Map

log_ai/

??? ap

20. Final One-Page Presentation Summary

* **Problem**: Enterprise security teams face log format fragmentation across Cisco, Fortinet, Suricata, and pfSense devices, leading to alert fatigue and unexplainable anomaly alerts.

* **S
with p